

AWS S3 → Lambda → Glue End-to-End Reference

This document provides a complete reference for an S3-triggered AWS Lambda function that starts an AWS Glue job. It includes console screenshots, Lambda and Glue code, and a detailed explanation of why the S3 object key includes prefixes such as **lz/**.

Why the S3 Object Key Includes the Prefix (lz/)

Amazon S3 does not have a real folder hierarchy. What appears as folders in the S3 console are simply prefixes that are part of the object key. When a file is uploaded to **s3://mith25/lz/aadharpancarddata.csv**, S3 stores it internally as a bucket name and a full key string.

Bucket: mith25

Key: lz/aadharpancarddata.csv

S3 event notifications always send the full object key, including prefixes. Lambda must use this exact key. Omitting the prefix will result in a **NoSuchKey** error.

AWS Console Reference Screenshots

Search

[Alt+S] Ask Amazon Q

EC2

Aurora and RDS

S3

Lambda

EMR

Amazon S3 > Buckets > mih25 > lz/

lz/

Copy S3 URI

Objects

Properties

Objects (2)

Copy S3 URI Copy URL Download Open L Delete Actions Create folder Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

	Name	Type	Last modified	Size	Storage class
	aadharpancarddata.csv	csv	December 13, 2025, 13:22:28 (UTC-05:00)	3.2 KB	Standard
	emp_manager_sal.csv	csv	December 13, 2025, 13:26:58 (UTC-05:00)	1.3 KB	Standard

Successfully updated the function `lambdareaddata`.

The test event "new2111" was successfully saved.

Notifications 0 0 0 5 0 0 0

Log output

The area below shows the last 4 KB of the execution log. [Click here](#) to view the corresponding CloudWatch log group.

```
START RequestId: 76fad902-462e-4dc9-9a27-4f2c3b937187 Version: $LATEST
Started glue job gluecsvread
JobRunId: jr_9344f6b8b546a1302fac180d829f0f041a2b2d80f0a725923f8ec55fc81a46f
S3 Object: s3://mih25/lz/emp_manager_sal.csv
END RequestId: 76fad902-462e-4dc9-9a27-4f2c3b937187
REPORT RequestId: 76fad902-462e-4dc9-9a27-4f2c3b937187 Duration: 158.61 ms Billed Duration: 159 ms Memory Size: 128 MB Max Memory Used: 93 MB
```

Test event Info

Delete CloudWatch Logs Live Tail Save Test

To invoke your function without saving an event, modify the event, then choose Test. Lambda uses the modified event to invoke your function, but does not overwrite the original event until you choose Save.

Test event action

Create new event Edit saved event

Invocation type

Synchronous Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

Asynchronous Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.

Event name

new2111

Event JSON

Format JSON

```
1 {
2   "Records": [
3     {
4       "s3": {
5         "bucket": {
6           "name": "mih25"
7         },
8         "object": {
9           "key": "lz/emp_manager_sal.csv"
10        }
11      }
12    ]
13  }
14 }
```


Lambda Code (S3 Trigger → Glue Job)

```
import json
import urllib.parse
import boto3

glue = boto3.client("glue")
GLUE_JOB_NAME = "your-glue-job-name"

def lambda_handler(event, context):
    if "Records" not in event:
        print("Non-S3 invocation")
        print(json.dumps(event))
        return {"statusCode": 400, "body": "Not an S3 event"}

    record = event["Records"][0]
    bucket = record["s3"]["bucket"]["name"]
    key = urllib.parse.unquote_plus(record["s3"]["object"]["key"])

    response = glue.start_job_run(
        JobName=GLUE_JOB_NAME,
        Arguments={"--bucket": bucket, "--key": key}
    )

    job_run_id = response["JobRunId"]

    print(f"Started Glue job: {GLUE_JOB_NAME}")
    print(f"JobRunId: {job_run_id}")
    print(f"S3 Object: s3://{bucket}/{key}")

    return {
        "statusCode": 200,
        "body": json.dumps({
            "message": "Glue job started",
            "job_name": GLUE_JOB_NAME,
            "job_run_id": job_run_id
        })
    }
```

Sample S3 Event Payload

```
{
  "Records": [
    {
      "s3": {
        "bucket": { "name": "mith25" },
        "object": { "key": "lz/emp_manager_sal.csv" }
      }
    }
  ]
}
```

AWS Glue Job Code (Reads Data Passed by Lambda)

```
import sys
from awsglue.utils import getResolvedOptions
from awsglue.context import GlueContext
from awsglue.job import Job
from pyspark.context import SparkContext

args = getResolvedOptions(sys.argv, ["JOB_NAME", "bucket", "key"])

bucket = args["bucket"]
key = args["key"]

sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)
job.init(args["JOB_NAME"], args)

s3_path = f"s3://{bucket}/{key}"

df = spark.read.option("header", "true").option("inferSchema", "true").csv(s3_path)

record_count = df.count()
df.printSchema()
df.show(truncate=False)

print(f"Glue Job processed file: {s3_path}")
print(f"Record count: {record_count}")

job.commit()
```

The Glue job receives the S3 bucket and key as arguments from Lambda. This allows the job to process exactly the file that triggered the Lambda invocation.